RAMDE - Engenharia Orientada a Requisitos e Modelos

Mestrado em Engenharia de Sistemas Computacionais Críticos

Lecture 02

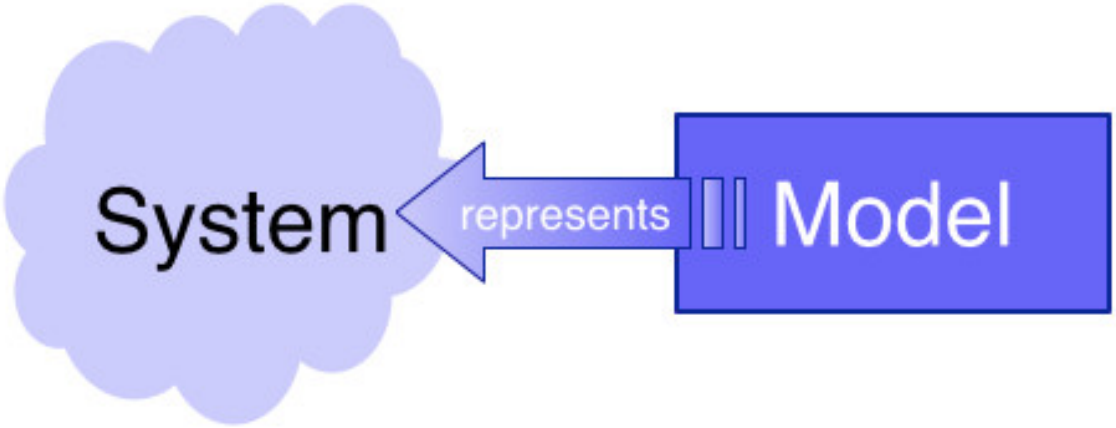
Introduction to MDE and EMF

Alexandre Bragança atb@isep.ipp.pt

Dep. de Engenharia Informática – ISEP

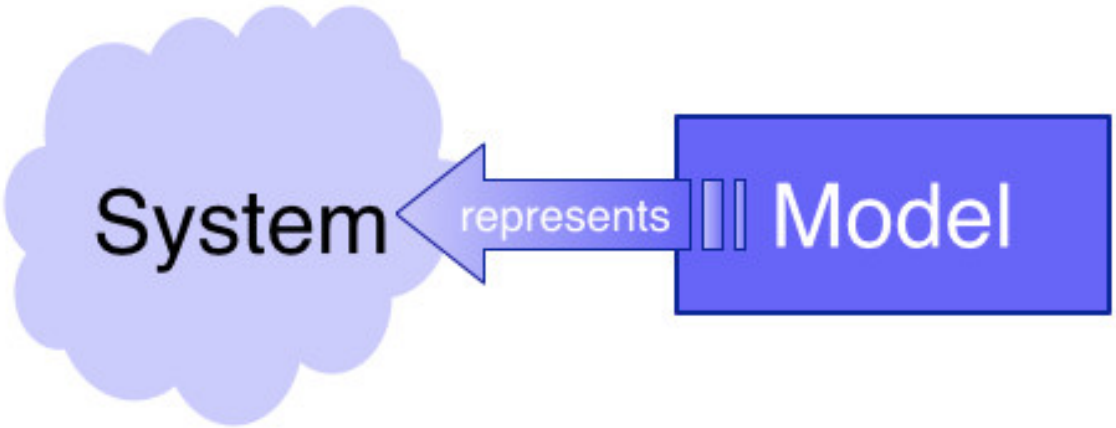
2026/2027

Abstractions and Models



- The human mind continuously re-works reality by applying cognitive processes
- **Abstraction**: capability of finding the commonality in many different observations:
  - generalize specific features of real objects (**generalization**)
  - classify the objects into coherent clusters (**classification**)
  - aggregate objects into more complex ones (**aggregation**)
- **Model**: a simplified or partial representation of reality (i.e., an abstraction), defined in order to accomplish a task or to reach an agreement.

Models: What is a Model?



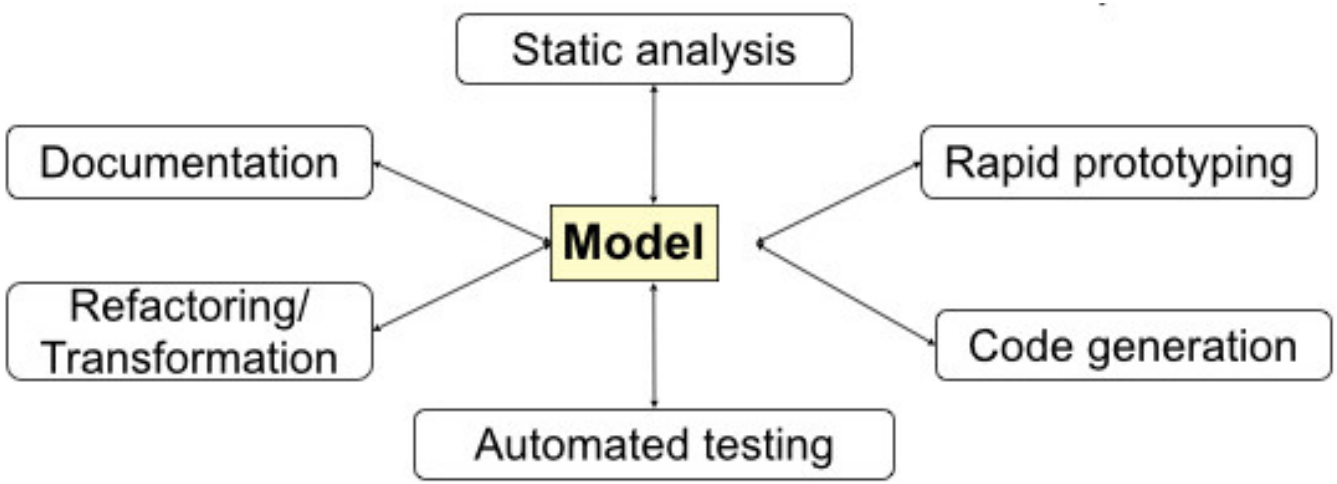
Mapping Feature	A model is based on an original (=system)
Reduction Feature	A model only reflects a (relevant) selection of the original's properties
Pragmatic Feature	A model needs to be usable in place of an original with respect to some purpose

Purposes:

- descriptive purposes
- prescriptive purposes

What is Model Engineering?

Model as the **central artifact** of software development



Related terms:

- Model Driven Engineering (MDE),
- Model Driven [Software] Development (MDD/MDSD),
- Model Driven Architecture (MDA)
- Model Integrated Computing (MIC)

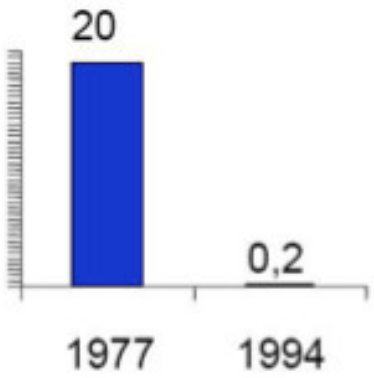
Why Model Engineering?

- **Increasing complexity of software**
  - Increasing basic requirements, e.g., adaptable GUIs, security, network capabilities, ...
  - Complex infrastructures, e.g., operating system APIs, language libraries, application frameworks
- **Software for specific devices**
  - Web browser, mobile phone, navigation system, video player, etc.

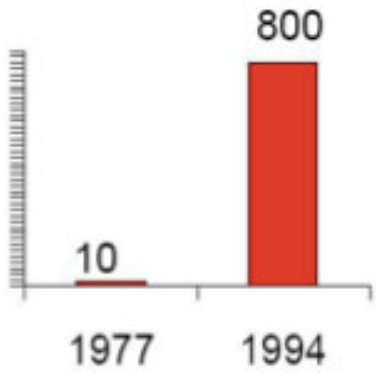
- **Technological progress ...**
  - Integration of different technologies and legacy systems, migration to new technologies
- **... leads to problems with software development**
  - Software finished too late
  - Wrong functionality realized
  - Software is poorly documented/commented
  - and can not be further developed, e.g., when the technical environment changes, business model/ requirements change, etc.

Why Model Engineering?

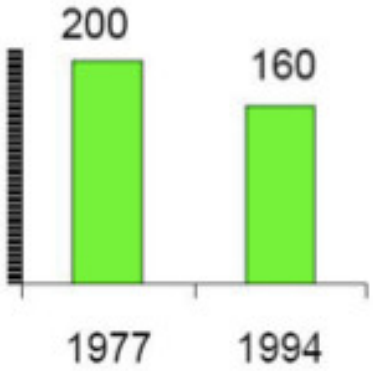
Quality problems in software development



Number of bugs per 1000 LOC



Program size (1000 LOC)



Resulting absolute bug count

Real quality improvements are only possible if the increase in program complexity is **overcompensated** !

(Average values, from Balzert 96)

How are we compared to 1994?

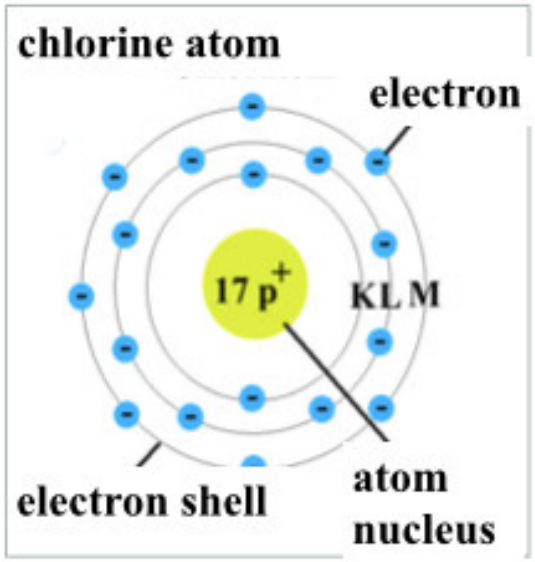
Why Model Engineering?

- **Traditional** usage of models in software development
  - **Communication** with customers and users (requirement specification, prototypes)
  - Support for software design, capturing of the **intention**
  - **Task specification** for programming
  - **Code visualization**
- What is the **difference** to Model Engineering?

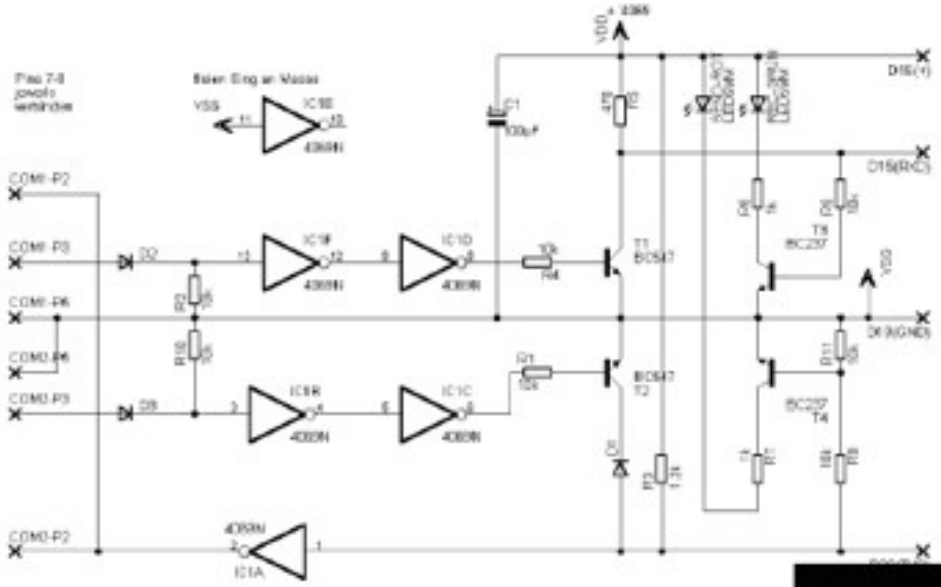
Usage of Models

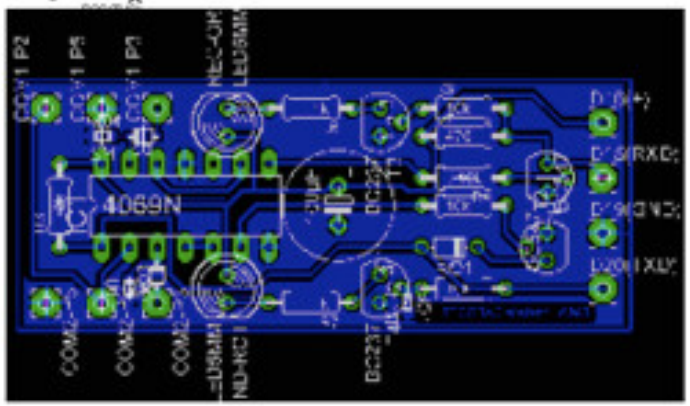
- Do not apply models as long as you have not checked the underlying **simplifications** and evaluated its **practicability**.
- Never mistake the **model** for the **reality**.
  - Attention: abstraction, abbreviation, approximation, visualization, ...





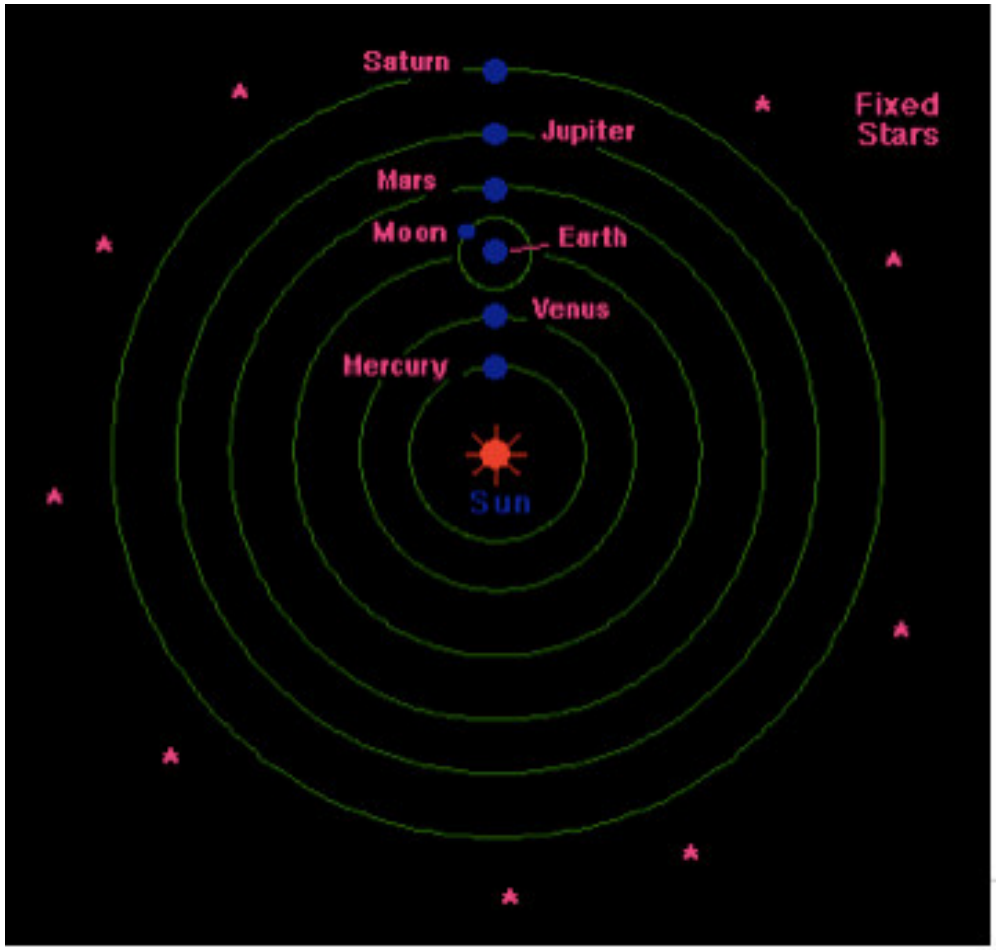
Constructive models (Example: Electrical Engineering)





Declarative models (Example: Astronomy)

Heliocentric model by Kopernikus



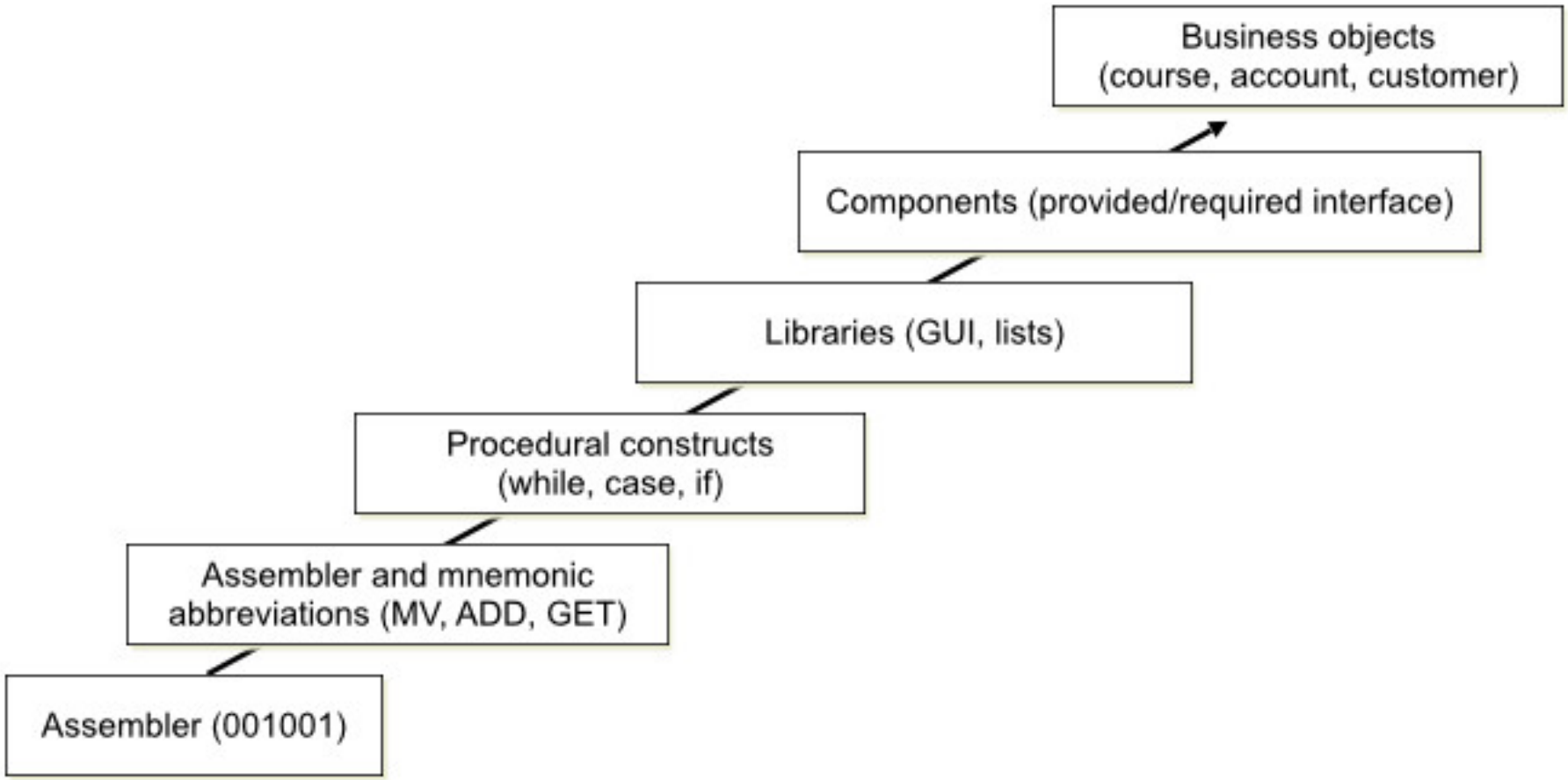
Application area of modeling



- **Models as drafts**
  - Communication of ideas and alternatives
  - Objective: modeling per se
- **Models as guidelines**
  - Design decisions are documented
  - Objective: instructions for implementation
- **Models as programs**
  - Applications are generated automatically
  - Objective: models are source code and vice versa

Increasing abstraction in software development

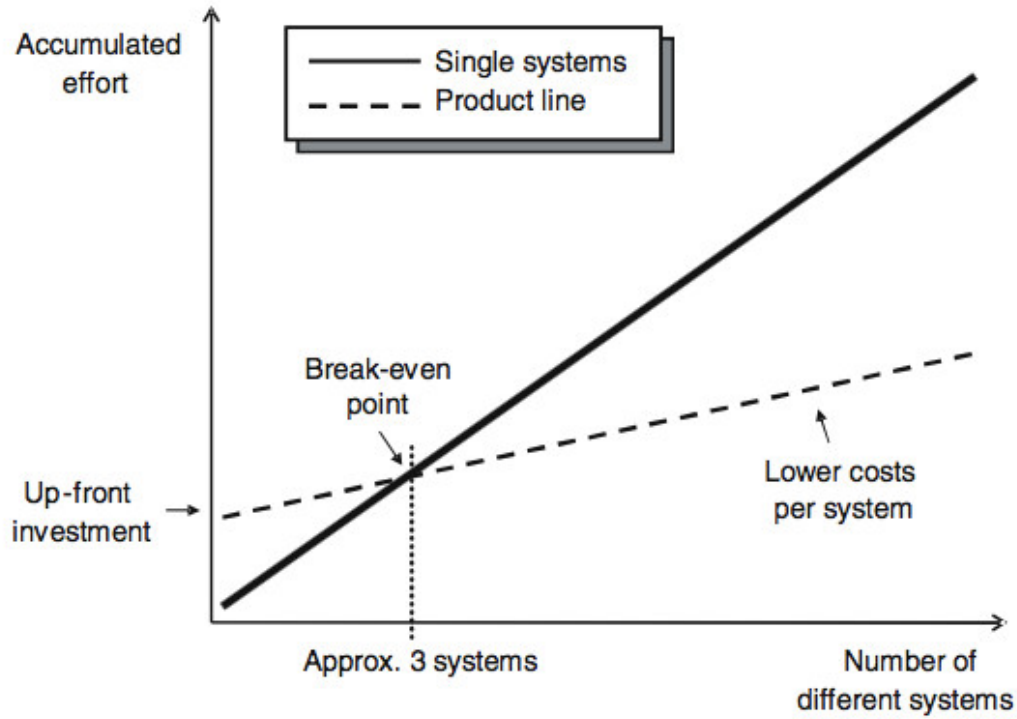
The **used artifacts of software development** slowly converge to the concepts of the **application area**. We have also being increasing how we are able to **reuse constructs in our software solutions**.
Such concepts can be **realized as domain-specific languages (DSL)**



Industrial Reuse -> Software Product Lines

A software product line is a set of software-intensive systems ***sharing a common, managed set of features*** *that satisfy the specific needs of a* ***particular market segment or mission*** *and that are developed from a* ***common set of core assets in a prescribed way***[1].

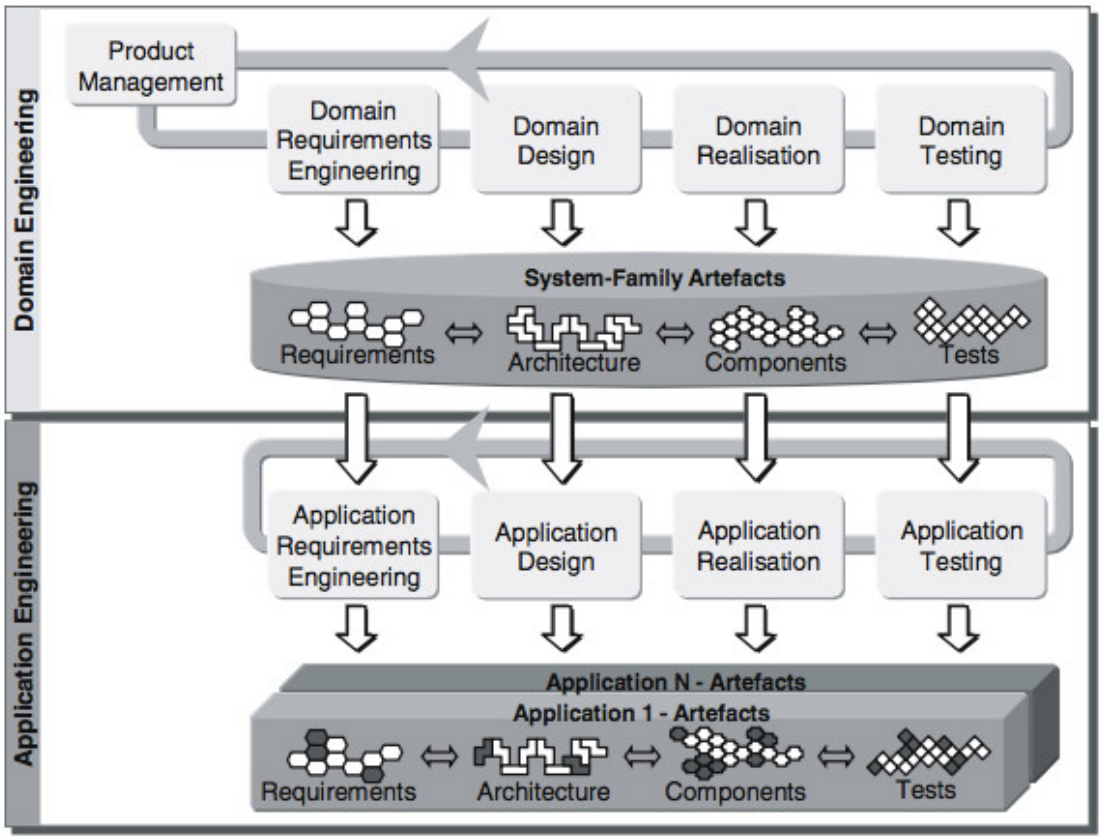
Figure: Economics of software product line engineering[2]



[1] From "Software Product Lines: Practices and Patterns", Paul Clements, Linda Northrop, Addison-Wesley, 2001

[2] From "Software Product Lines in Action", Frank van der Linden et al, Springer, 2007

Figure: The two-life-cycle model of software product line engineering[3]



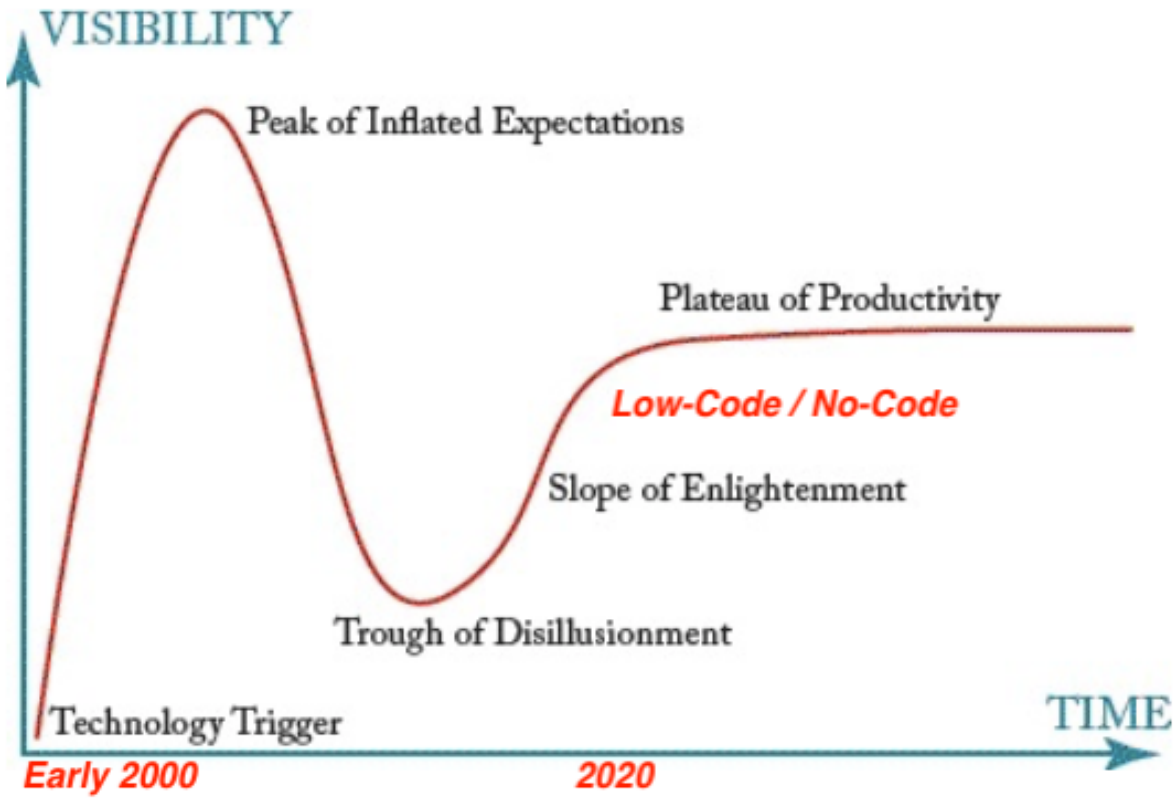
[3] From "Software Product Lines in Action", Frank van der Linden et al, Springer, 2007

Software Product Lines Fundamental Principles

- **Variability management**: individual systems are considered as variations of a common theme. This variability is made explicit and must be systematically managed.
- **Business-centric**: software product line engineering aims at thoroughly connecting the engineering of the product line with the long-term strategy of the business.
- **Architecture-centric**: the technical side of the software must be developed in a way that allows taking advantage of similarities among the individual systems.
- **Two-life-cycle approach**: the individual systems are developed based on a software platform. These products – as well as the platform – must be engineered and have their individual life-cycles.

DSL, MDE, SPL: Are Not These Old Solutions from 2000?

Figure: Technology Hype Cycle



Low-Code / No-Code seems to be the democratization of the approach and the really **enabler of MDE productivity for the masses...**

DSL, MDE, SPL: Who is using these approaches globally?

The **Software Product Line Conference**[4] has nominated a series of systems for its hall of fame. Included in this list are companies such as:

- Boing
- Bosch
- Ericsson
- General Motors
- HP
- Lucent
- Nokia
- Philips
- Siemens
- Toshiba
- US Army

Note: These are all systems that have been documented and presented at the SPLC conference.

[4] `http://splc.net/hall-of-fame/`

DSL, MDE, SPL: What tools to use?

DSLs, MDE and SPLs do not require specific tools. It is possible to adopt any of these approaches **using general and widely used software engineering tools**.

However, there are also specific tools oriented to support such approaches. Some of the major tools are: [5]

- EMF - Eclipse Modeling Framework (`https://www.eclipse.org/modeling/`). The reference tool solution for MDE and DSL development. Several tools built around EMF:
  - ATL - ATL Transformation Language (`http://www.eclipse.org/atl/`)
  - Acceleo (`http://www.eclipse.org/acceleo/`)
  - Xtext (`http://www.eclipse.org/Xtext/`)
  - Sirius (`https://www.eclipse.org/sirius/`)
- MPS - Meta Programming System (`https://www.jetbrains.com/mps/`)
- Modeling SDK for Visual Studio - Domain-Specific Languages (`https://docs.microsoft.com/en-us/visualstudio/modeling/modeling-sdk-for-visual-studio-domain-specific-languages?view=vs-2017`)

[5] This list is not complete.

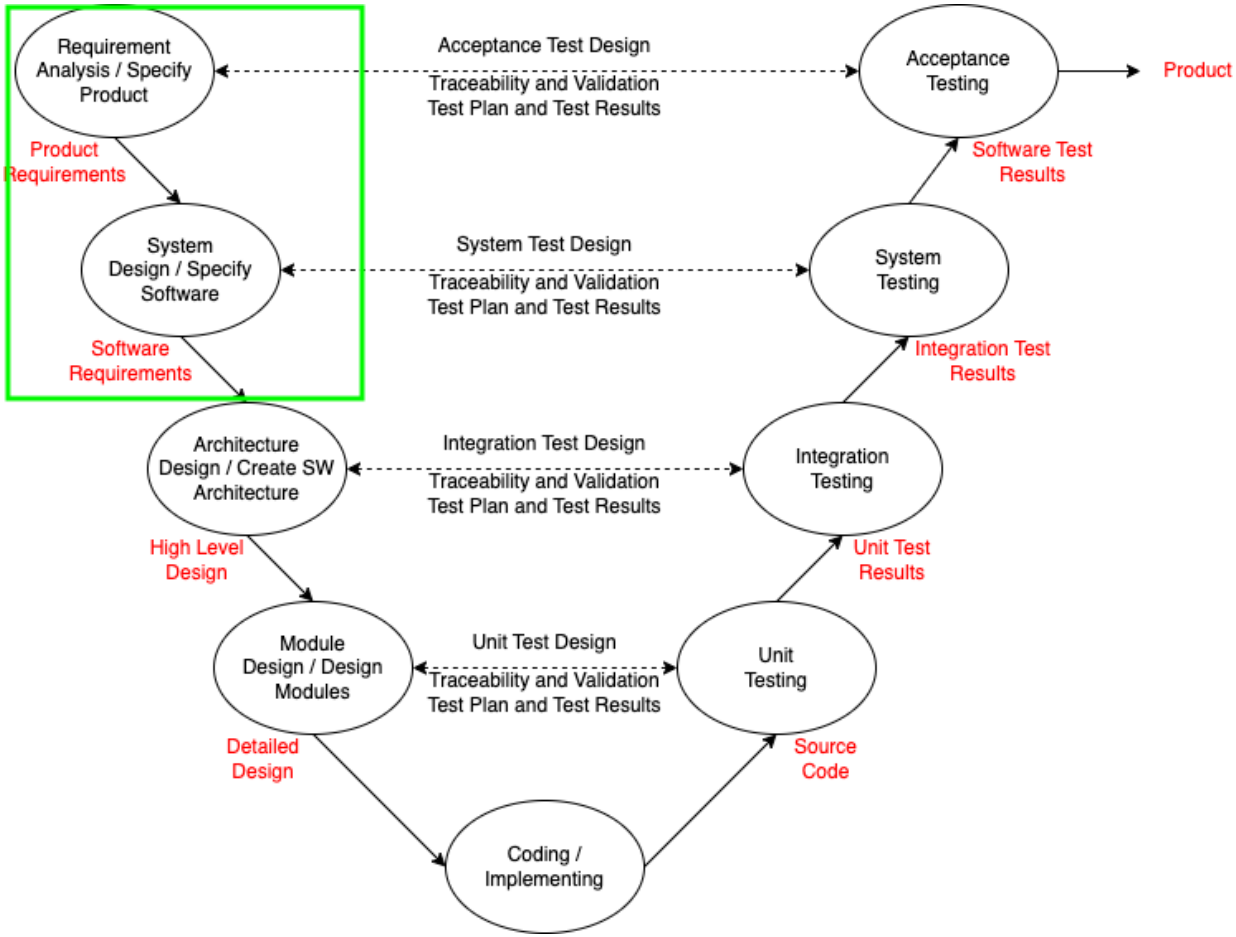
Requirements and Requirements Modeling

Requirements Modeling - A Simple Example - Modeling in Eclipse

Overview

- Lets focus on models to capture requirements
- We start by a summary of requirements concepts
- We then move to illustrate how MDE can be used to model requirements
- We will use Eclipe, EMF - Eclipse Modeling Framework, and Eclipse Xtext

Requirements



Definitions (1/2)

Requirement[6]

1. A condition or capability needed by a user **to solve a problem or achieve an objective**.
2. A condition or capability that must be met or possessed by a system or system component **to satisfy a contract, standard, specification, or other formally imposed documents**.
3. A documented representation of a condition or capability as in (1) or (2).

Stakeholder

1. A stakeholder of a system is a person or an organization that **has** an (direct or indirect) **influence on the requirements of the system**.

[6] 610.12-1990 - IEEE Standard Glossary of Software Engineering Terminology

Definition (2/2)

Requirements Engineering

1. Requirements engineering is a systematic and disciplined approach to the specification and management of requirements with the following goals:
   1. **Knowing the relevant requirements**, achieving a consensus among the stakeholders about these requirements, **documenting them** according to given standards, and **managing them systematically**
   2. Understanding and documenting the stakeholders' desires and needs, thus specifying and managing requirements to minimize the risk of delivering a system that does not meet the stakeholders' desires and needs

Four core activities of requirements engineering

- Elicitation
- Documentation
- Validation and negotiation
- Management

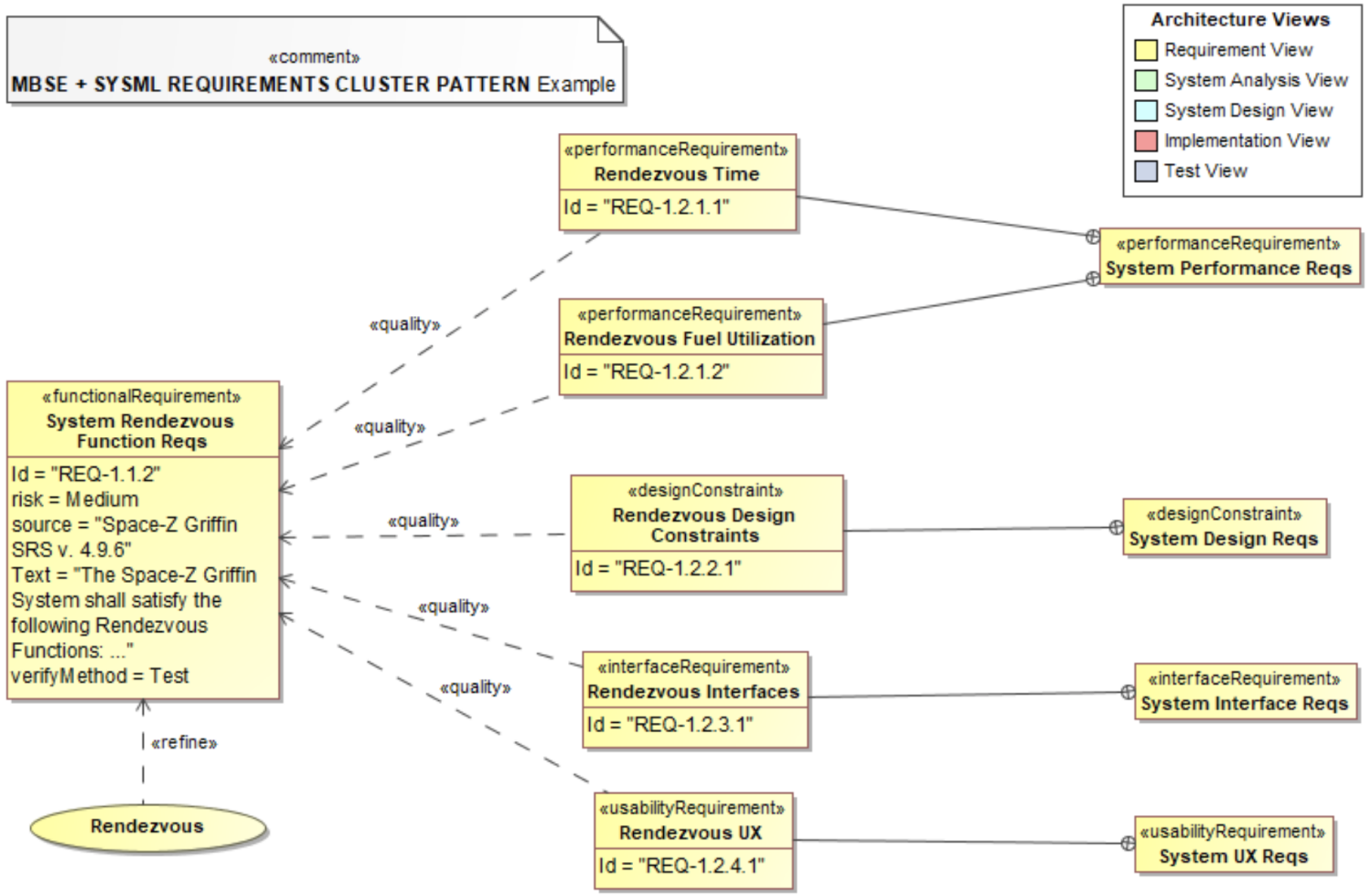
Requirement Types

Generally, one can distinguish between three types of requirements:[7]

- Functional requirements
  - Define the functionality that the system to be developed offers.
  - Usually, these requirements are divided into **functional requirements, behavioral requirements, and data requirements**
- Quality requirements
  - Define desired qualities of the system to be developed and often influence the system architecture more than functional requirements do.
  - Typically, quality requirements are about the **performance, availability, dependability, scalability, or portability of a system**.
  - Requirements of this type are frequently classified as non-functional requirements.
- Constraints
  - Constraints cannot be influenced by the team members.
  - Requirements of this type can constrain the system itself (e.g., "The system shall be implemented using web services") or the development process ("The system shall be available on the market no later than the second quarter of 2012").
  - In contrast to functional and quality requirements, **constraints are not implemented, they are adhered to** because they merely limit the solution space available during the development process.

[7] A number of different classifications of requirements are used in practice. For example, there are a number of classifications suggested by several standards, e.g., CMMI [SEI 2006] or SPICE [ISO/IEC 15504-5].

Requirements Models - The Case of SysML



SysML (Systems Modeling Language) is a standard modeling language used for systems engineering, defined by the Object Management Group (OMG) and International Council on Systems Engineering (INCOSE).

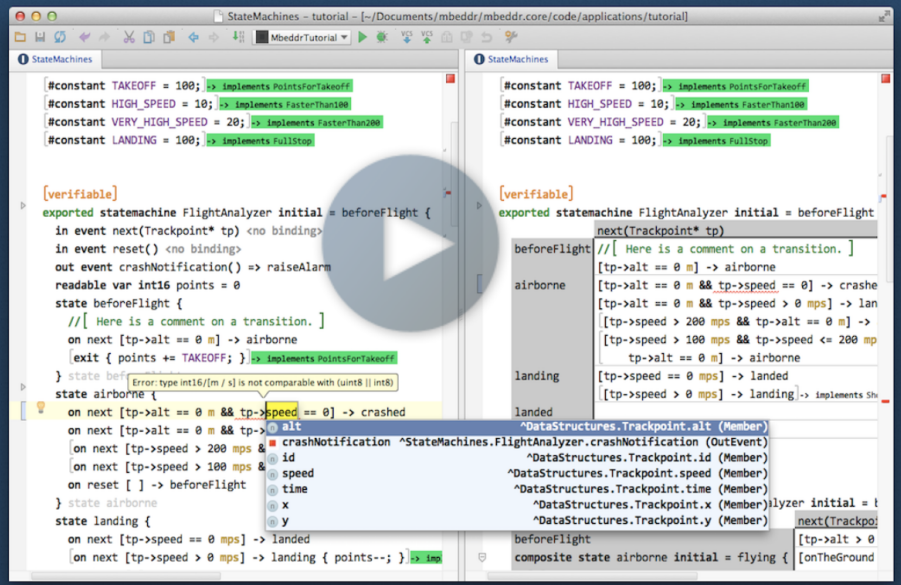
Requirements Models - The Case of MBeddr

mbeddr News Benefits Download Platform Learn FAQ Screencasts Support Team MPS

ENGINEERING THE FUTURE OF EMBEDDED SOFTWARE

Boosting productivity and quality by using extensible DSLs, flexible notations and integrated verification tools.

Download



WHAT IS MBEDDR?

mbeddr is a set of integrated and extensible languages for embedded software engineering, plus an IDE. It supports implementation, testing, verification and process aspects. It integrates with command-line build tools and integration servers, as well as file-based version control systems.

mbeddr has support for requirements and product line definition, software documentation, implementation in C and C extensions such as state machines physical units or interfaces and components, as well as testing, mocking, as well as formal verification.

mbeddr comes with a state-of-the-art IDE including syntax coloring, code completion, go to definition, realtime type checks, quick fixes, refactorings, customizable find-usages, automated synchronization between related parts of the code, version control integration and debugging.

MBeddr is tool aimed at modeling and building embedded software by using domain-specific languages and MDE. It includes a DSL to model requirements.

The RAMDE Approach

In RAMDE we will

- Use an hybrid approach
- We will use standards (i.e., EMF as an implementation of EMOF[8]) to create the concepts and rules of our modeling languages
- Our modeling languages will be inspired by standards, but simplified
- We will implement our modeling languages with Eclipse Xtext (textual syntax) and PlantUML (graphical syntax)

[8] https://www.omg.org/mof/

Lab Class First Example

Requirements Modeling - A Simple Example - Modeling in Eclipse

Consider a **light switch** where one can turn on and off the light as well as change the brightness of the light (3 levels) by using the on button. The off position turns off the light.

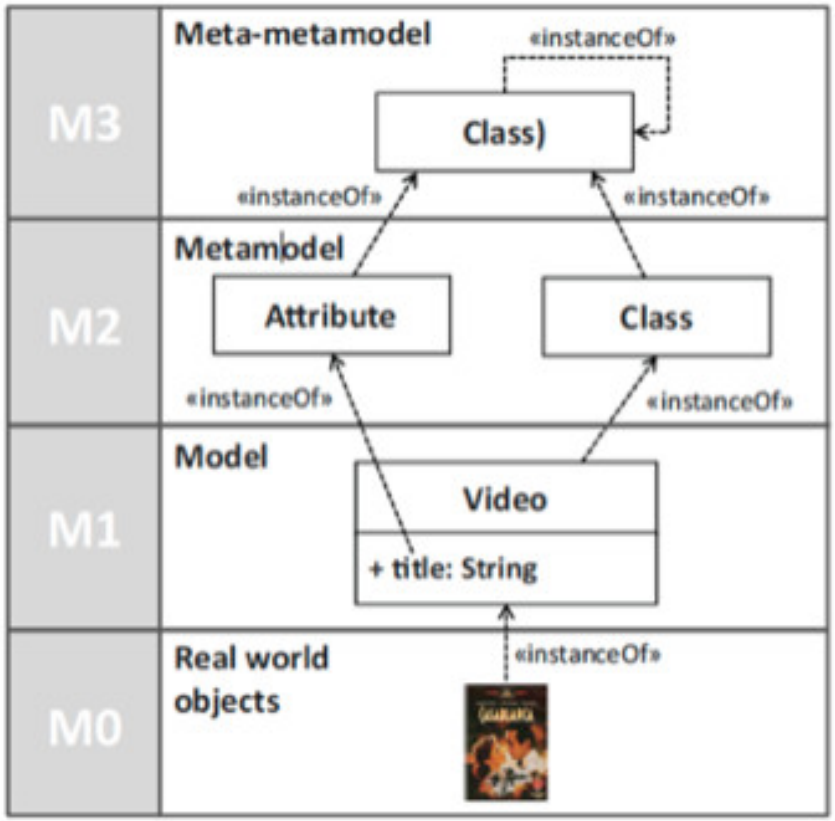
Light Switch Requirements

- REQ-01
  - Power Control
  - The system shall enable or disable power delivery to the load based on user input.
- REQ-02
  - Multi-Level Brightness
  - The system shall support 3 distinct brightness levels (Low: 30%, Medium: 60%, High: 100%).
- REQ-03
  - Single-Button Brightness Cycle
  - The ON button shall cycle sequentially through brightness levels upon successive presses when already powered ON.
- REQ-04
  - OFF State Reset
  - Powering OFF the light shall reset the next turn-on state to Level 1 (Low).

Preamble: Fundamentals of MDE: Models and Metamodels (1/2)

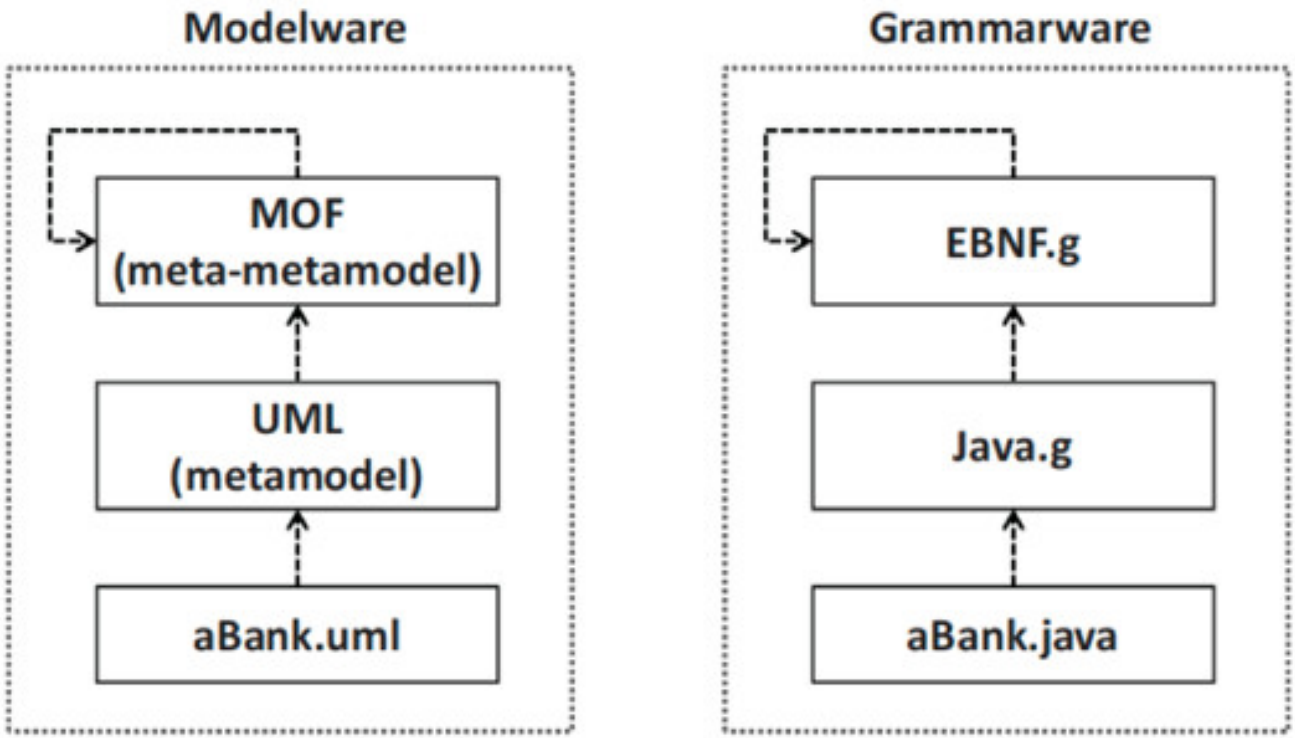
In MDE:

- A **Model** is defined following the "constraints/rules" of its **Metamodel**
- **Metamodel** = yet another abstraction, highlighting properties of the model itself
- A metamodel is also a model
- Every model has its metamodel
- Metamodels can be used for:
  - defining new languages
  - defining new properties or features of existing information (metadata)

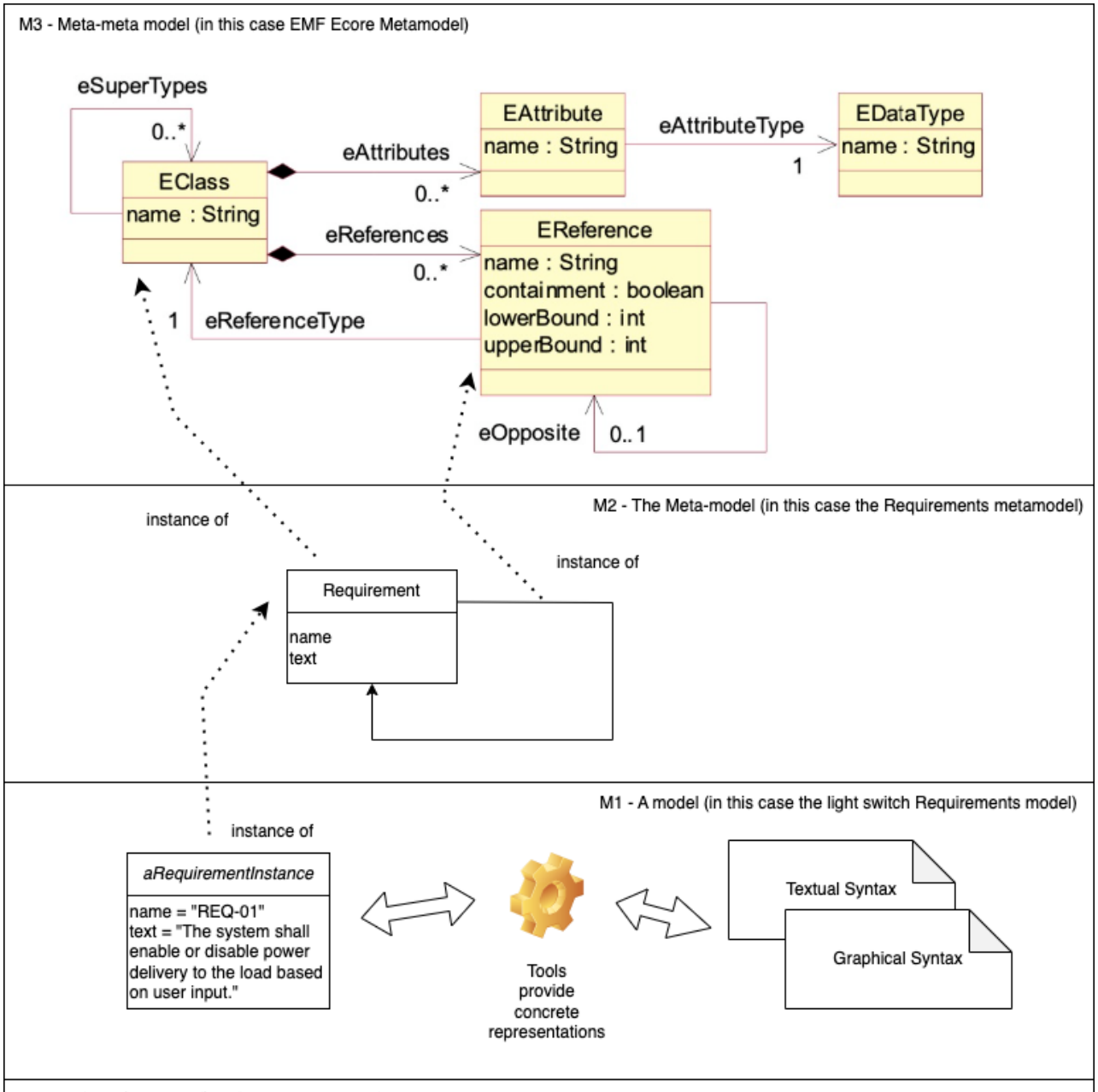


Preamble: Fundamentals of MDE: Models and Metamodels (2/2)

- MDE concepts are "similar" to concepts of programming languages and grammars
- The relation between metamodel and models is similar to the relation between a program in a programming language and the grammar of that programming language



Preamble: Fundamentals of MDE: The Modeling Levels



Meta-Metamodel: Ecore - EMF Metamodel: Core Concepts

To model the M2 model we use the M3 modeling concepts of Ecore (the EMF Metamodel).

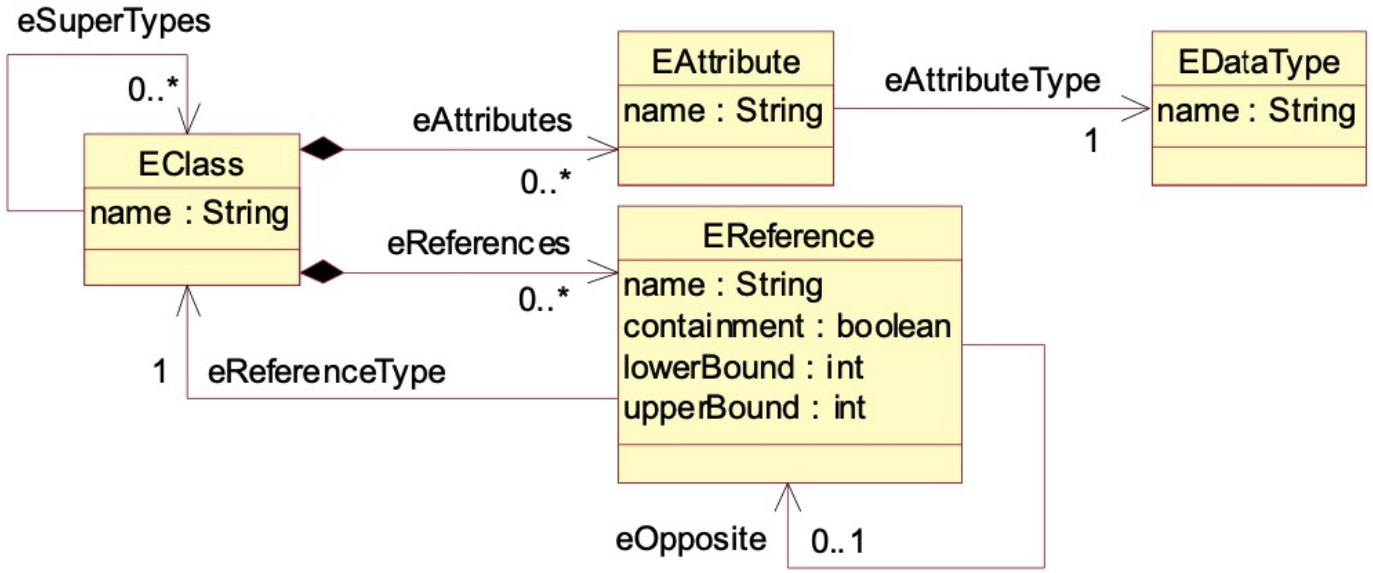


Figure: Excerpt of the Ecore Metamodel

- These are the main concepts used in Ecore to specify all possible models!
- Ecore is, in fact, an implementation of the EMOF metamodel from OMG (EMOF = Essential Meta Object Facility)
- For a full diagram of the Ecore Metamodel see
  `http://www.kermeta.org/docs/org.kermeta.ecore.documentation/build/html.chunked/Ecore-MDK/ch02.html`

Demonstration: The Light Switch Example

1. Lets build a modeling environment for Requirements
   - Lets build a Requirements metamodel (Eclipse EMF)
   - Lets build a Textual Language for Requirements (Eclipse Xtext)
   - Lets build a Graphical Diagram Projection for Requirements models (Eclipse Xtext + PlantUML)
2. Lets use the modeling environment to model the requirements of the light switch